Lab -7: Experiment No: 7

AIM

To study Truffle, establish a local Ethereum blockchain using Ganache, and deploy and interact with a Solidity smart contract for blockchain-based college event certificate issuance and verification.

THEORY

1. Truffle
Truffle is a development framework used to create, compile, test, and deploy Ethereum smart contracts. It provides project structure, configuration files, migration scripts, and testing facilities that simplify blockchain application development.
Its major features include:

- Smart contract compilation and deployment.
- Automated testing of contracts.
- Management of deployment scripts.
- Configuration for different blockchain networks.

2. Ganache
Ganache is a local Ethereum blockchain used for developing and testing decentralized applications. It provides a simulated blockchain environment with pre-funded accounts and test Ether.
Its important features are:

- Creation of test accounts with Ether.
- Local transaction execution and block mining.
- Inspection of transaction history and block details.
- Testing of smart contracts without using real cryptocurrency.

3. MetaMask
MetaMask is an Ethereum-compatible cryptocurrency wallet that manages blockchain accounts and enables interaction with decentralized applications.
In this experiment, MetaMask is connected to the local Ganache network. A Ganache account can be imported into MetaMask using its private key, allowing transactions to be signed and submitted to the local blockchain.

4. Remix IDE
Remix IDE is a browser-based environment for developing Solidity smart contracts. It provides facilities for writing code, compiling contracts, deploying them to a selected blockchain, and calling their functions.
For this experiment, Remix is used to compile and deploy the certificate management contract through MetaMask.

5. Local Blockchain
A blockchain is a ledger in which transactions are grouped into blocks and linked using cryptographic techniques. A local blockchain reproduces the essential transaction and contract-execution workflow without requiring deployment on a public network.

Ganache serves as the local blockchain in this experiment. Changes to the contract state, such as issuing or revoking certificates, are recorded through transactions.

6. Smart Contract

A smart contract is a program deployed on a blockchain that executes predefined operations when its functions are called.

The contract developed in this experiment represents a **Blockchain-Based College Event Certificate Issuance and Verification System**. It allows an authorized account to issue certificates, retrieve their details, verify their authenticity, and revoke invalid certificates.

7. Certificate Structure

A Solidity structure (struct) groups related certificate information into a single record.

```
struct Certificate {
    uint256 certificateId;
    string studentName;
    string eventName;
    string certificateCID;
    bytes32 certificateHash;
    address issuer;
    bool revoked;
}
```

The fields store the certificate identifier, student's name, event name, IPFS content identifier, cryptographic hash, issuer's address, and revocation status.

8. Mapping

A mapping stores associations between keys and values. In this contract, a certificate ID is associated with its corresponding certificate record.

```
mapping(uint256 => Certificate) public certificates;
```

This enables certificate records to be accessed using their unique IDs.

9. msg.sender and require()

msg.sender identifies the blockchain address that calls a function. It is used to record the issuer's address and restrict sensitive operations to the authorized account.

The require() statement validates conditions before an operation is executed. If a condition is not satisfied, the transaction is reverted. It can prevent duplicate certificate IDs, reject nonexistent records, and restrict unauthorized operations.

10. Events

Solidity events create entries in transaction logs when specified operations occur. Events such as CertificateIssued and CertificateRevoked help applications track certificate-related activity.

11. IPFS and Certificate Hash

The InterPlanetary File System (IPFS) can store certificate documents outside the blockchain and provide a Content Identifier (CID) for retrieving them. The CID can be recorded in the smart contract.

A cryptographic hash acts as a digital fingerprint of a document. Comparing a document's calculated hash with the stored hash helps determine whether the document matches the recorded version.

For this local demonstration, the CID and hash can be supplied as test values. Actual IPFS storage and document-hash calculation require additional application logic.

12. Certificate Revocation

A certificate may need to be invalidated if it was issued incorrectly. The contract therefore maintains a Boolean revoked field. It is initially false and becomes true when the authorized account revokes the certificate.

A revoked certificate should fail the contract's authenticity verification check.

DIAGRAMS

Diagram 1: Blockchain Development Environment

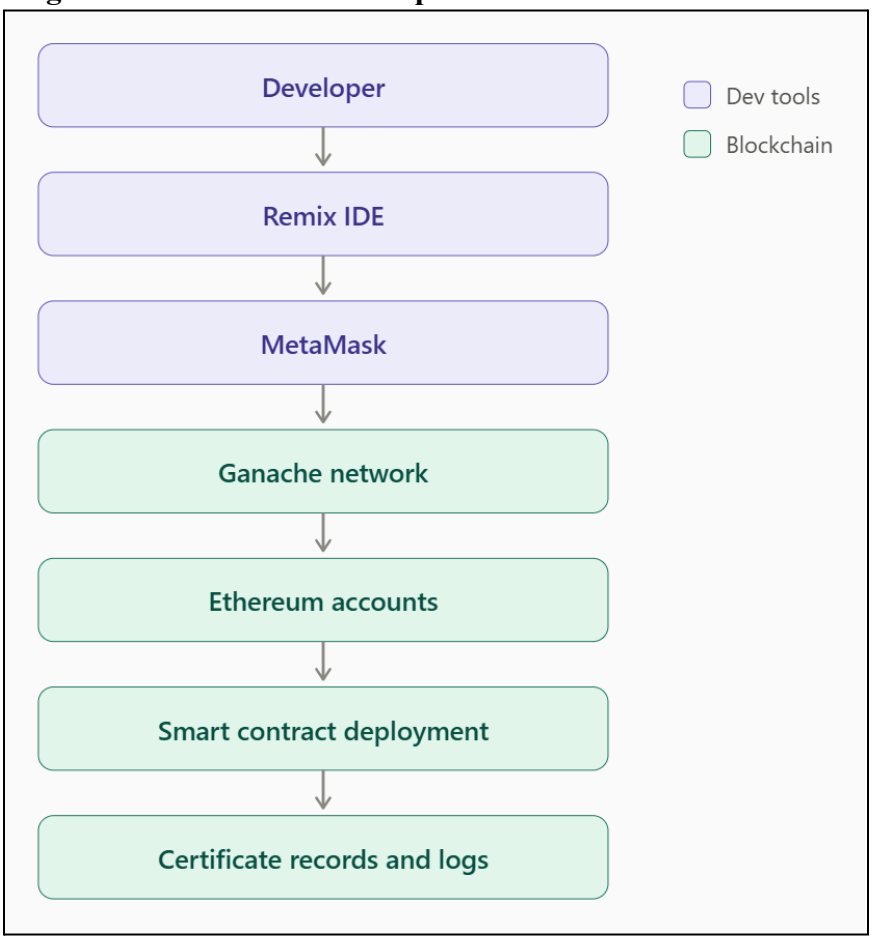
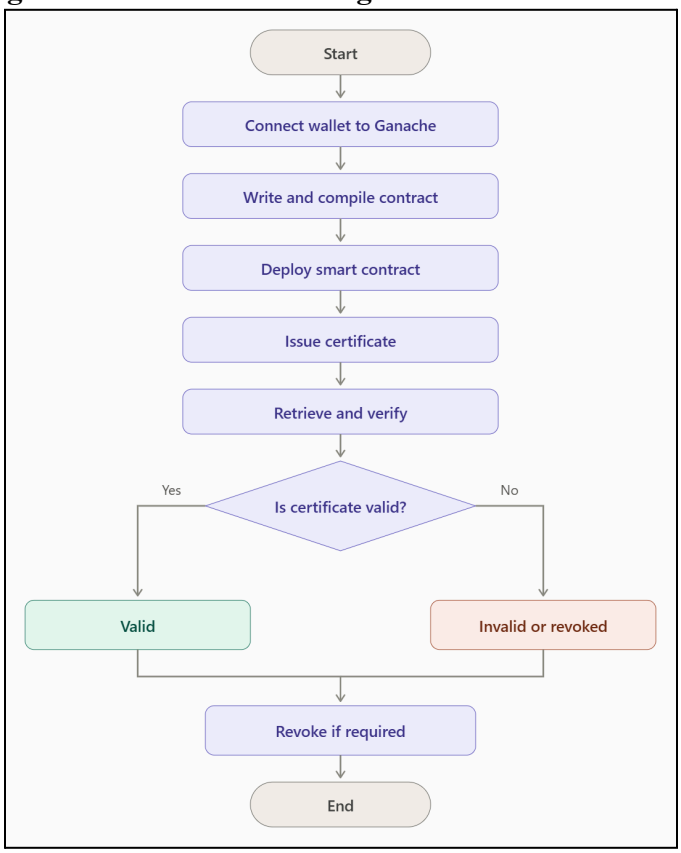


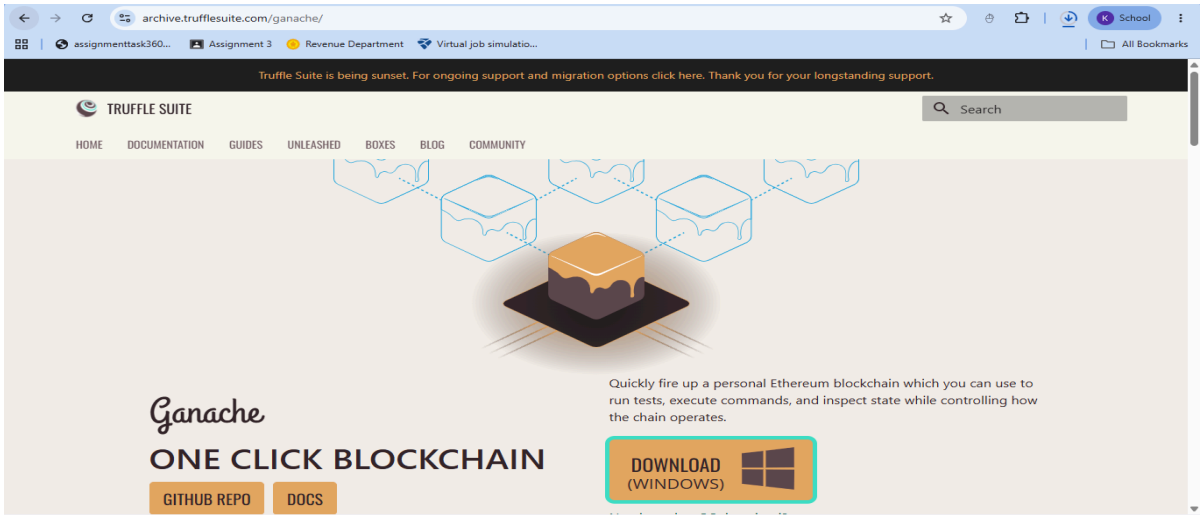
Diagram 2: Certificate Management Workflow

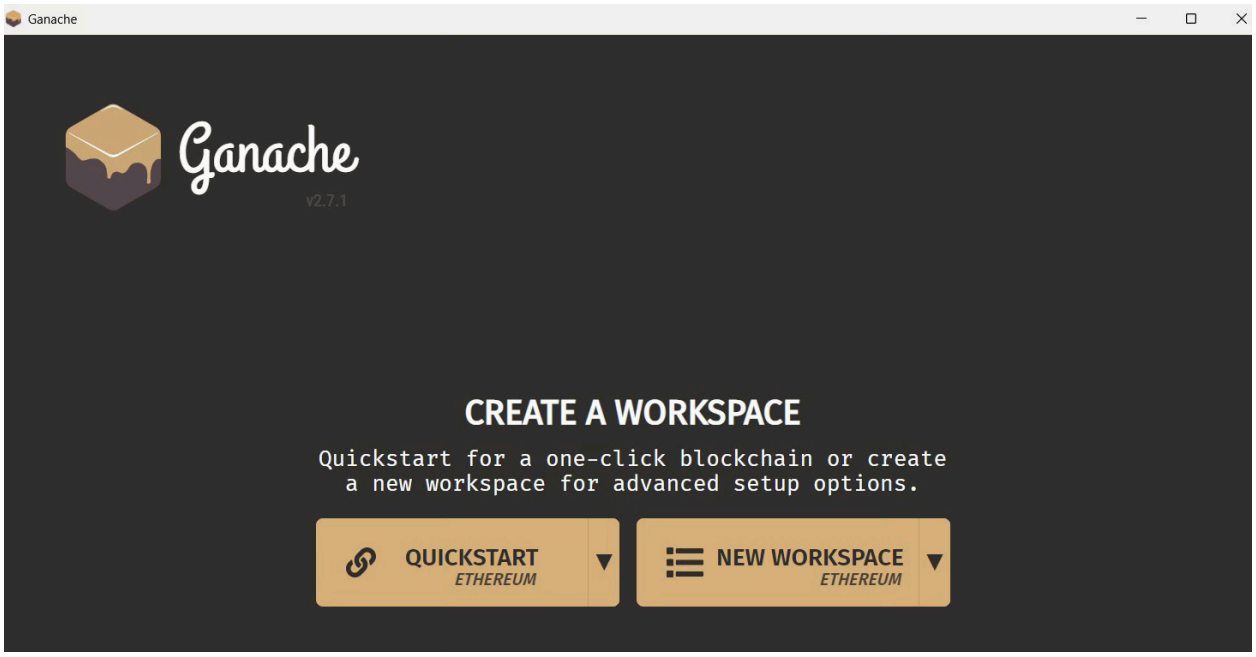


PROCEDURE

Step 1: Start Ganache

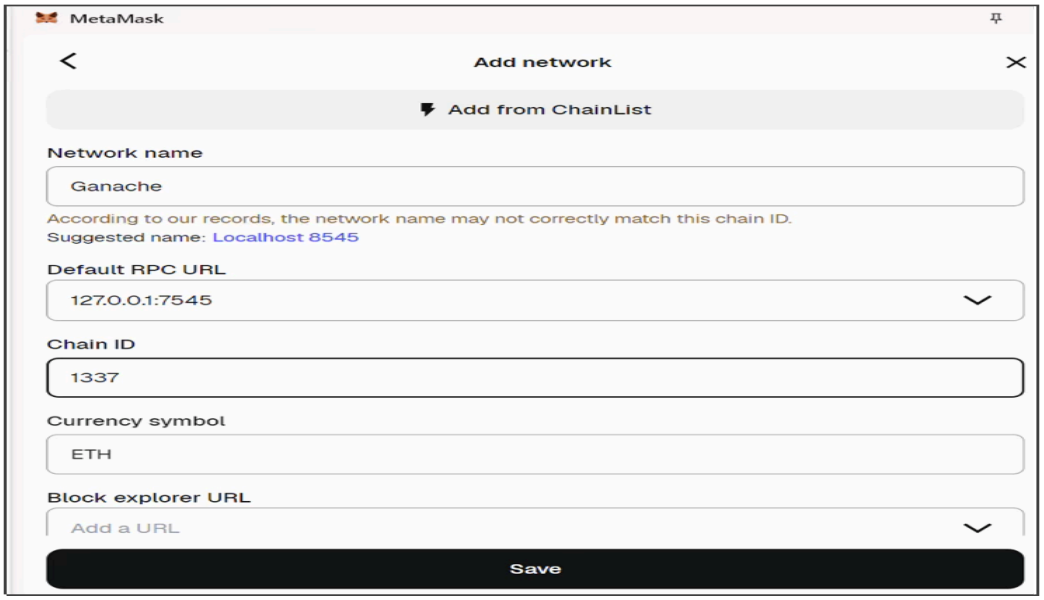
1. Install and open Ganache.
2. Select **Quickstart Ethereum** to create a local Ethereum blockchain.
3. Note the RPC server URL, network ID or chain ID, and available accounts.
4. Open the account details to view the test account's private key.
5. Keep Ganache running throughout the experiment

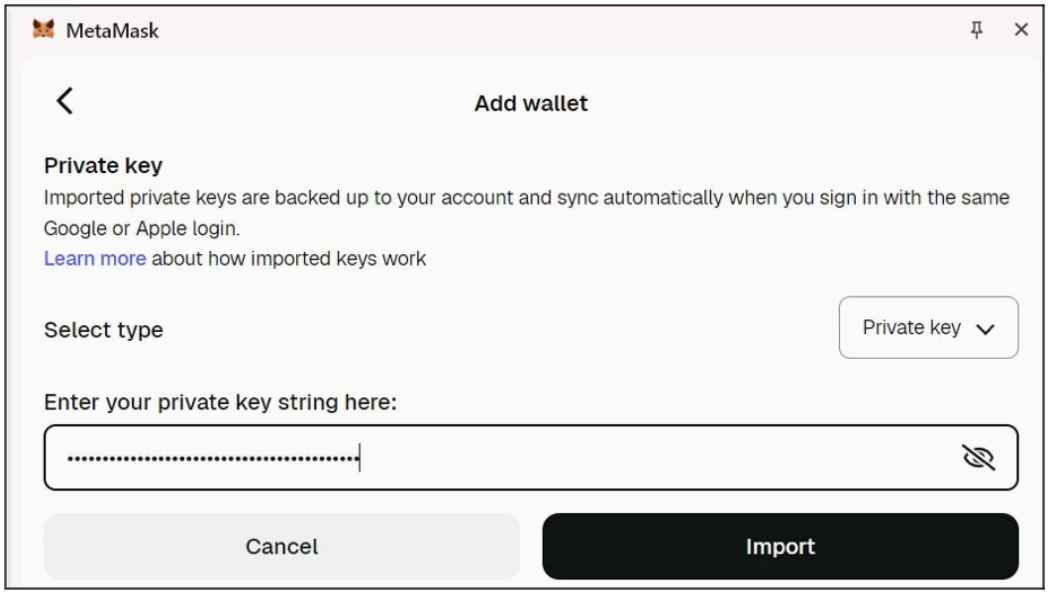


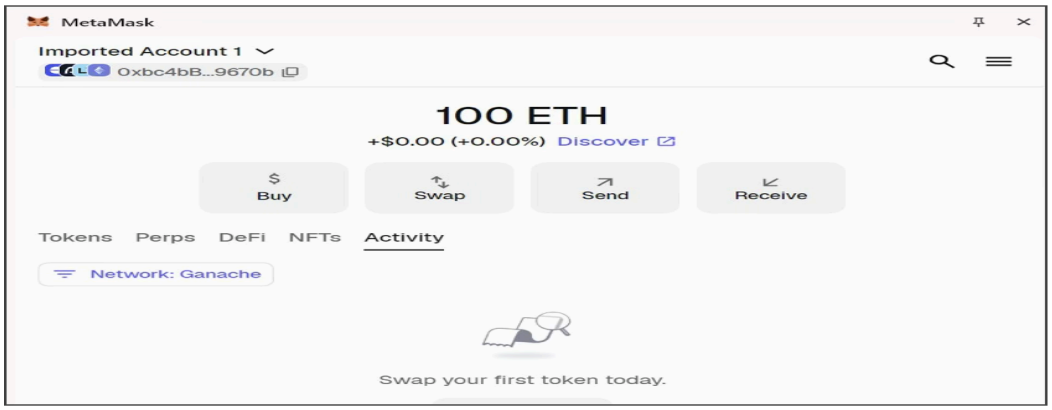


Step 2: Connect MetaMask to Ganache

1. Open the MetaMask browser extension.
2. Select the option to add a custom network.
3. Enter the RPC URL displayed in Ganache, such as http://127.0.0.1:7545 or the actual URL shown by your installation.
4. Enter the network name as Ganache Local.
5. Enter the correct chain ID displayed by Ganache.
6. Save the network configuration.
7. Import a Ganache account into MetaMask using its private key.
8. Select the imported account and verify that its displayed balance matches the local test account.







Step 3: Open Remix IDE

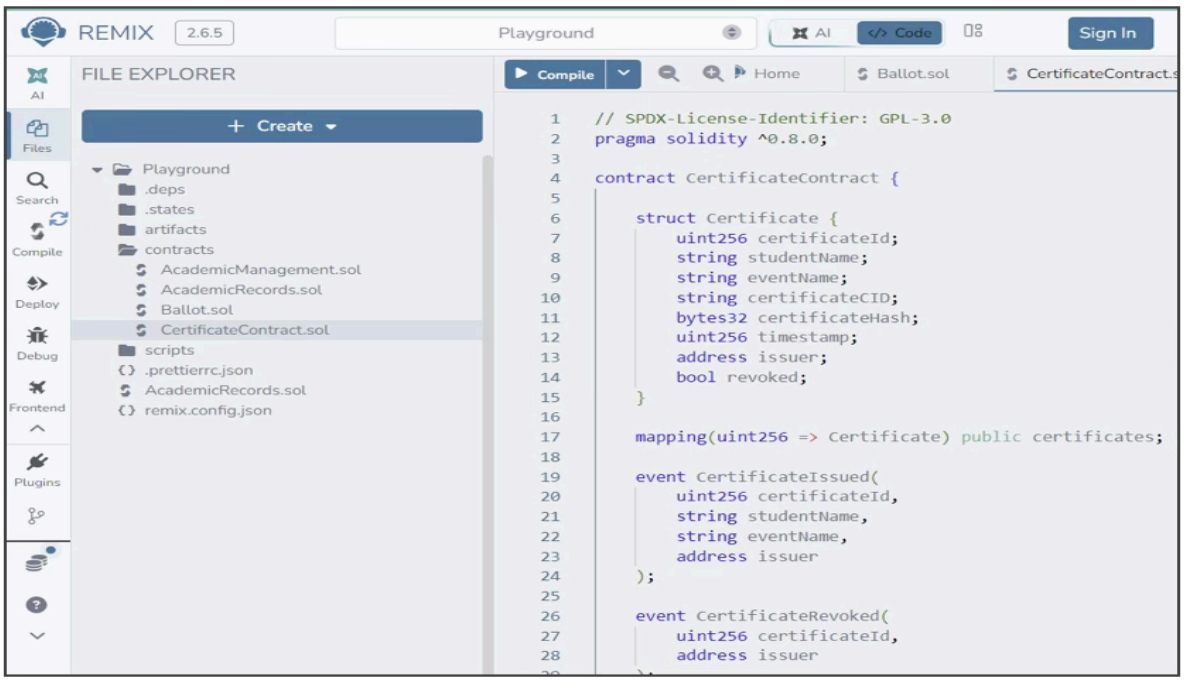
1. Open Remix IDE at https://remix.ethereum.org/.
2. Create a new Solidity file named CertificateManager.sol.
3. Enter the smart contract code given below.
4. Save the file.

Step 4: Write the Smart Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract CertificateManager {
   address public owner;
   struct Certificate {
      uint256 certificateId;
      string studentName;
      string eventName;
      string certificateCID;
      bytes32 certificateHash;
      address issuer;
      bool revoked;
   }
```

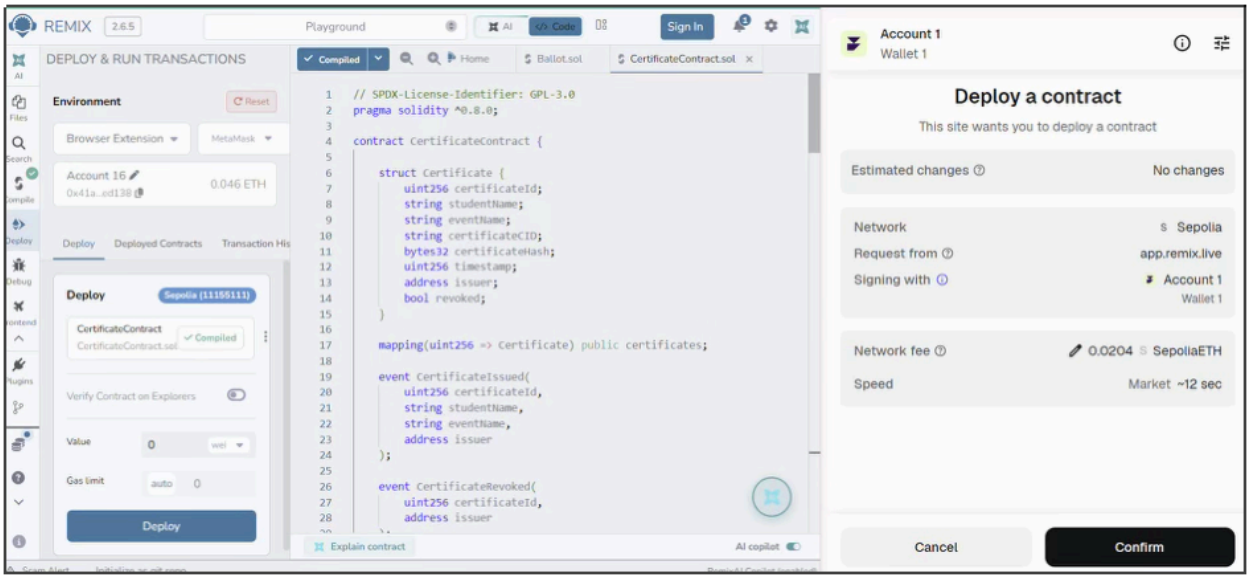
```
    mapping(uint256 => Certificate) public certificates;
    event CertificateIssued(
        uint256 certificateId,
        string studentName,
        string eventName
    );
    event CertificateRevoked(uint256 certificateId);
    modifier onlyOwner() {
        require(msg.sender == owner, "Not authorized");
        _;
    }
    constructor() {
        owner = msg.sender;
    }
    function issueCertificate(
        uint256 _certificateId,
        string memory _studentName,
        string memory _eventName,
        string memory _certificateCID,
        bytes32 _certificateHash
    ) public onlyOwner {
        require(_certificateId != 0, "Invalid certificate ID");
        require(
            certificates[_certificateId].certificateId == 0,
            "Certificate already exists"
        );
        certificates[_certificateId] = Certificate({
            certificateId: _certificateId,
            studentName: _studentName,
            eventName: _eventName,
            certificateCID: _certificateCID,
            certificateHash: _certificateHash,
            issuer: msg.sender,
            revoked: false
        });
        emit CertificateIssued(
            _certificateId,
            _studentName,
            _eventName
        );
    }
    function getCertificate(
        uint256 _certificateId
    ) public view returns (Certificate memory) {
        require(
            certificates[_certificateId].certificateId != 0,
```

```
            "Certificate does not exist"
        );
        return certificates[_certificateId];
    function verifyCertificate(
        uint256 _certificateId,
        bytes32 _certificateHash
    ) public view returns (bool) {
        Certificate storage c = certificates[_certificateId];
        return (
            c.certificateId != 0 &&
            c.certificateHash == _certificateHash &&
            !c.revoked
    }
    function revokeCertificate(
        uint256 _certificateId
    ) public onlyOwner {
        require(
            certificates[_certificateId].certificateId != 0,
            "Certificate does not exist"
        );
        require(
            !certificates[_certificateId].revoked,
            "Certificate already revoked"
        );
        certificates[_certificateId].revoked = true;
        emit CertificateRevoked(_certificateId);
    }
```



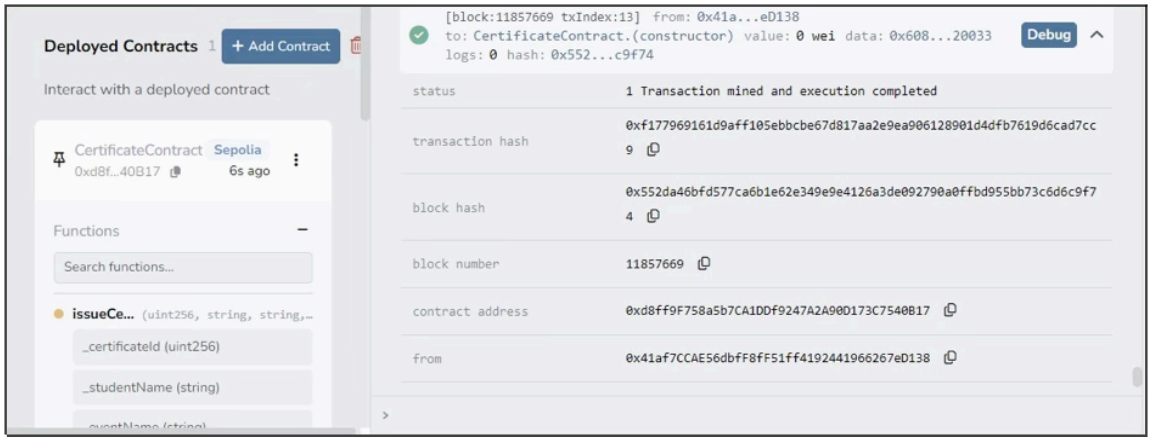
Step 5: Compile the Contract

1. Open the Solidity Compiler panel in Remix.
2. Select a compiler version compatible with Solidity ^0.8.20.
3. Click **Compile CertificateManager.sol**.
4. Check that no compilation errors are displayed.
5. Open the compilation details to confirm that the contract has been generated successfully.



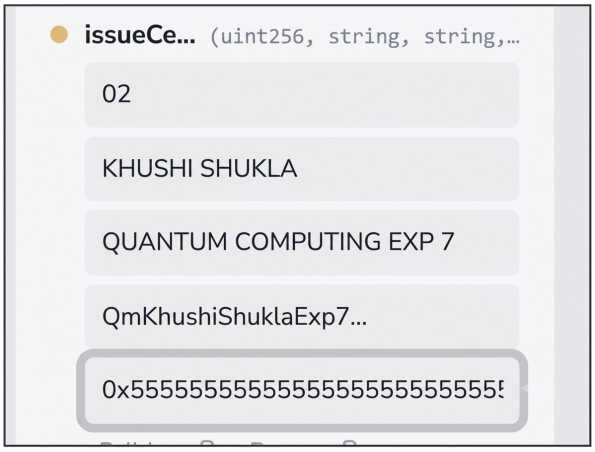
Step 6: Deploy the Contract

1. Open the **Deploy & Run Transactions** panel.
2. Select **Injected Provider - MetaMask** as the environment. The exact label may vary by Remix version.
3. Approve the connection request in MetaMask.
4. Confirm that MetaMask is using the Ganache network and the intended account.
5. Select CertificateManager from the contract list.
6. Click **Deploy**.
7. Confirm the deployment transaction in MetaMask.
8. Wait for the transaction to be mined.
9. Expand the deployed contract under **Deployed Contracts**.



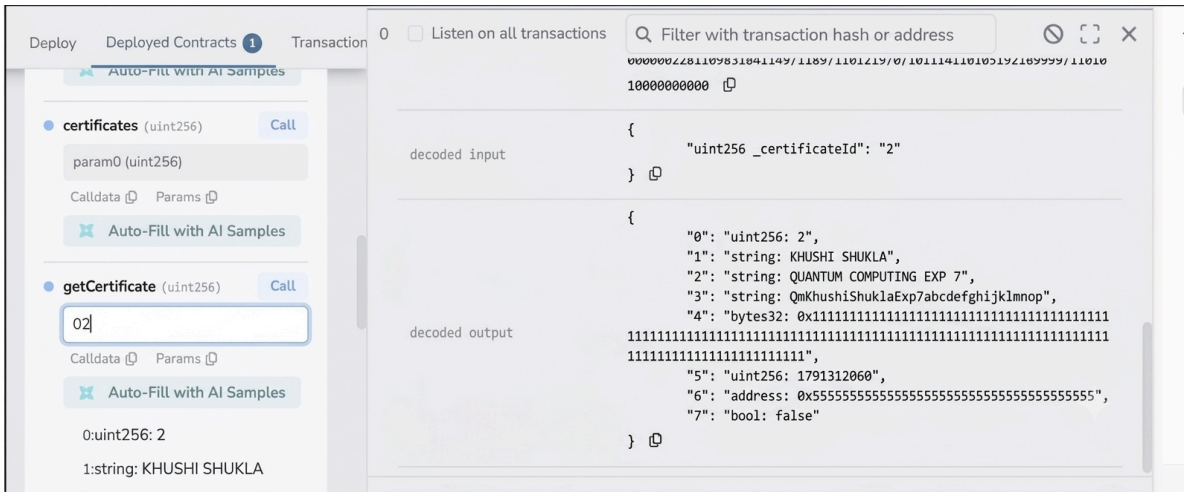
Step 7: Issue Certificate 1

Use the issueCertificate function to issue the first certificate.



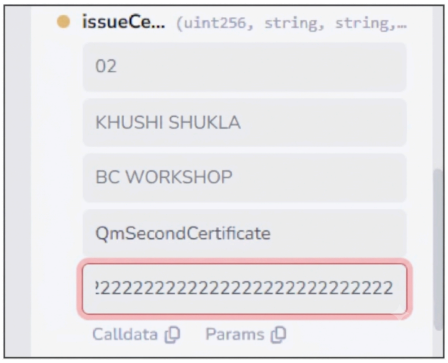
Step 8: Retrieve and Verify Certificate 1

1. Expand getCertificate.
2. Enter certificate ID 1001.
3. Execute the function and inspect the returned fields.
4. Expand verifyCertificate.
5. Enter certificate ID 1001 and the same hash used during issuance.
6. Execute the verification function.



Step 9: Issue and Verify Certificate 2

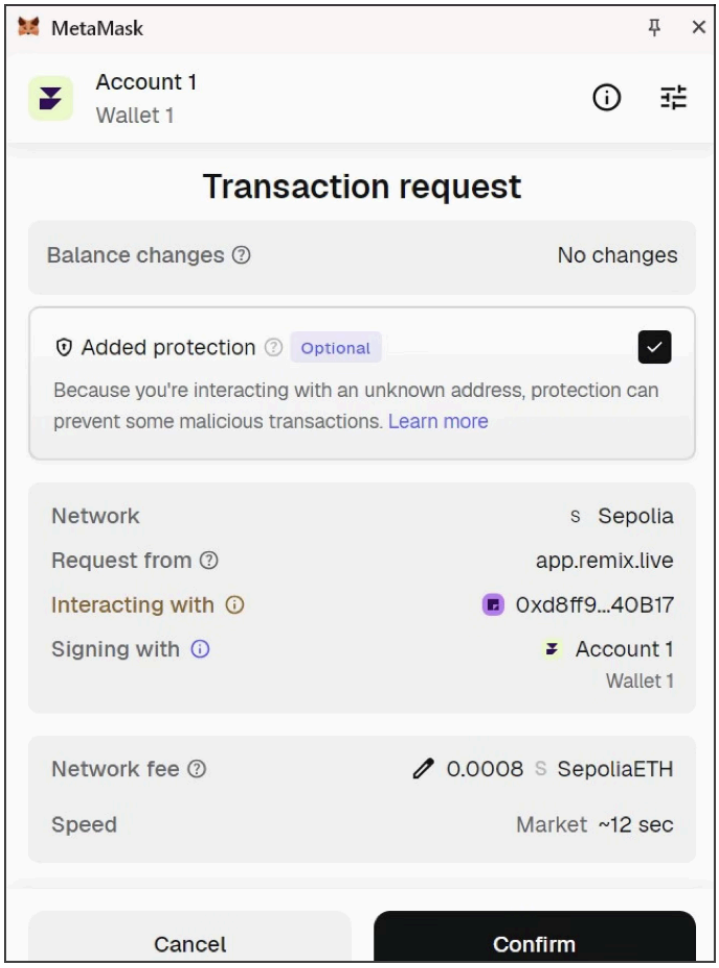
Repeat the issuance process with a different certificate ID and record.

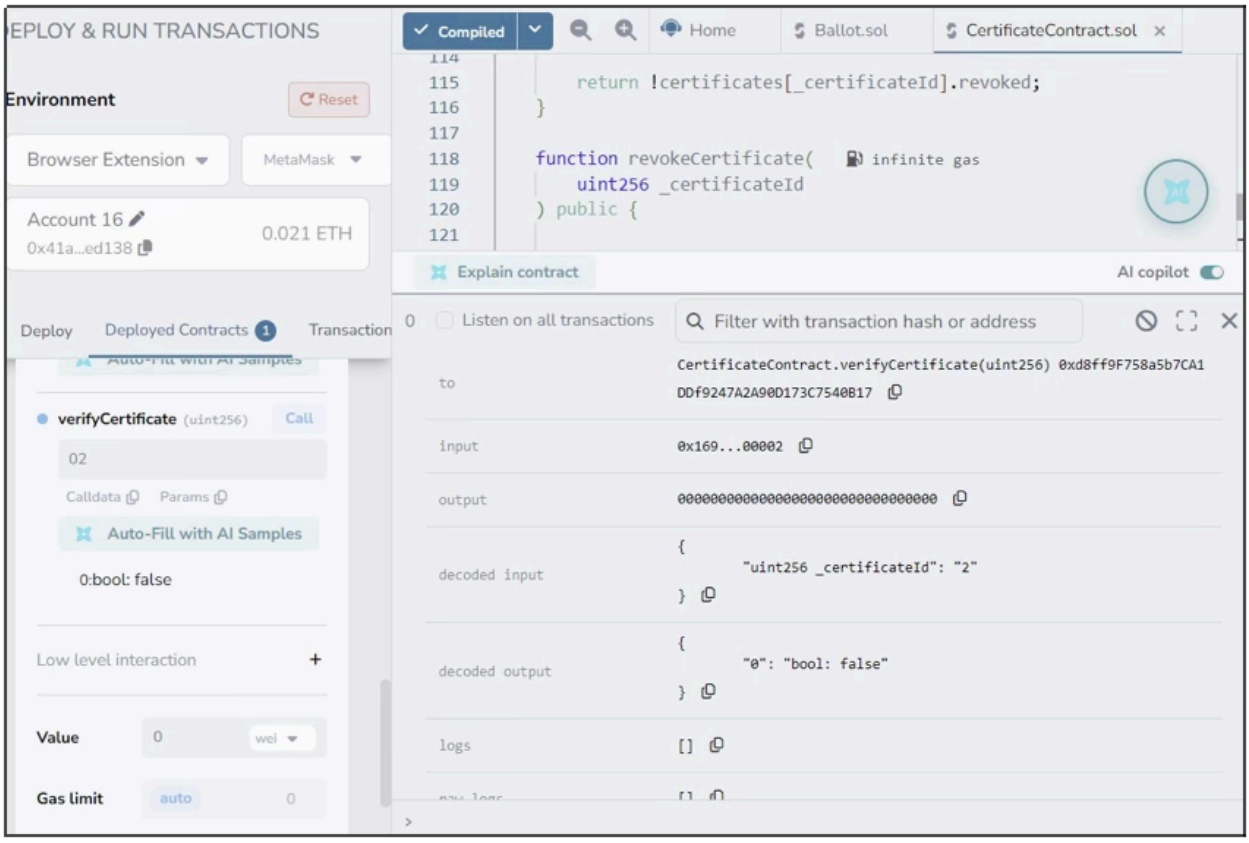




Step 10: Revoke Certificate 2

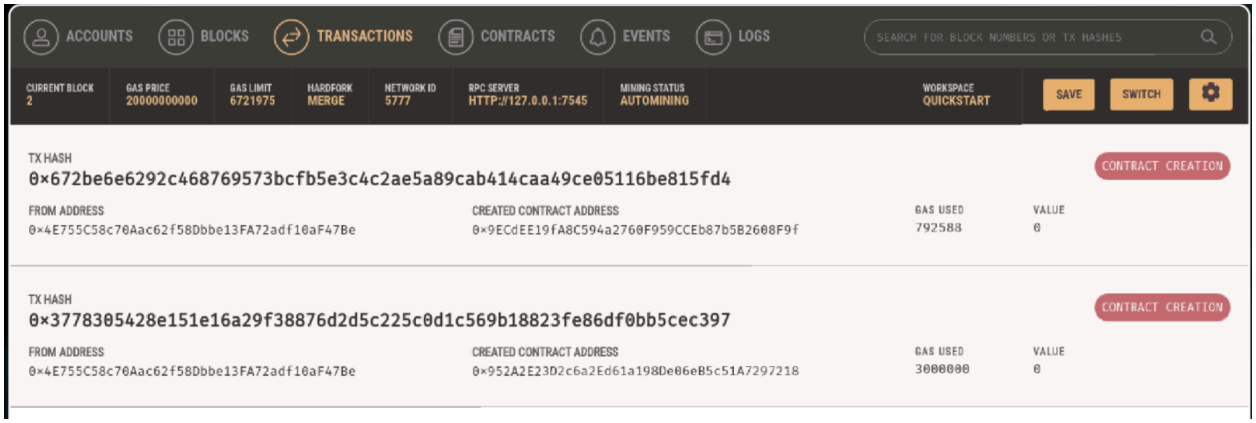
1. Expand the revokeCertificate function.
2. Enter certificate ID 1002.
3. Execute the function and approve the transaction in MetaMask.
4. Wait for the transaction to be mined.
5. Call getCertificate again with ID 1002 and inspect the revoked field.
6. Call verifyCertificate with ID 1002 and its original hash.





Step 11: Inspect Ganache Transactions

1. Return to Ganache.
2. Open the transaction list.
3. Inspect the deployment, issuance, and revocation transactions.
4. View the relevant transaction details and blocks.



CONCLUSION

Ganache, MetaMask, and Remix IDE enabled local smart contract deployment and testing. The certificate management system demonstrated secure certificate verification using blockchain records, hashing, access control, and revocation without a public Ethereum network.